

Linux System Calls and Process Management

1. How Linux Handles System Calls Internally

System calls are the interface between user-space applications and the Linux kernel. When a program executes a system call (like open, read, or write), it triggers a software interrupt or a specific CPU instruction (syscall on x86_64). The processor switches from user mode to kernel mode, saving the user context and executing the corresponding system call handler in the kernel. The kernel validates arguments, performs the requested operation using kernel data structures, and returns the result to user space before switching back to user mode.

2. What Happens Inside the Kernel When You Execute a Command in the Shell

When a user types a command in the shell, the shell parses the command and searches for the executable in directories listed in the PATH variable. It then calls `fork()` to create a new process and `exec()` to replace the child process's memory space with the executable's code. The kernel loads the binary into memory, sets up the process environment, and schedules it for execution. When the process terminates, the shell uses `wait()` to collect its exit status.

3. Role of init and Its Evolution to systemd

The init process (PID 1) is the first process started by the Linux kernel during boot. It initializes the system, spawns system processes, and manages runlevels or services. In modern systems, systemd replaces init, offering parallelized startup, service dependency management, logging, and cgroup-based process tracking. Unlike traditional init, which used shell scripts, systemd uses declarative unit files and is event-driven for faster and more reliable booting.

4. Interaction of `fork()`, `exec()`, and `wait()`

`fork()` creates a new child process by duplicating the parent's memory space. `exec()` replaces the current process image with a new program, effectively running a new executable in the same process ID. `wait()` is used by the parent to pause until the child finishes execution, allowing it to retrieve the child's exit status. Together, these three calls form the foundation of process creation and control in Unix-like systems.

5. Handling of Zombie and Orphan Processes

A zombie process occurs when a child finishes execution but its parent hasn't yet called `wait()` to collect its status. It remains in the process table as a 'defunct' entry. Once the parent calls `wait()`, the zombie is removed. An orphan process occurs when its parent terminates before the child. The orphan is then adopted by the init (or systemd) process, which automatically calls `wait()` to prevent zombies.

6. Process States in Linux

Linux processes transition through several states:

- R (Running): Actively executing or ready to run.
- S (Sleeping): Waiting for an event or I/O operation.
- D (Uninterruptible Sleep): Waiting for hardware or kernel event; cannot be interrupted.
- T (Stopped): Suspended by a signal or

debugger. • Z (Zombie): Completed execution but not yet reaped by parent. • I (Idle): Kernel threads not doing useful work.